Project Report

In today's fast-paced world, hospitals and healthcare systems require efficient and reliable tools to manage patient information, emergency cases, treatment history, and doctor scheduling. This project aims to implement a basic Hospital Management System using core data structures learned in the course. By applying concepts such as linked lists, stacks, priority queues, and hash tables, we developed a simulation of a real-world hospital environment. The project not only enhances our coding skills in Java but also demonstrates the practical application of data structures to solve real-life problems effectively.

1- Problem Statement

The primary objective of the Hospital Management System is to design a system that manages patient records, treatment histories, emergency cases, and doctor assignments using appropriate data structures. This system helps simulate real hospital operations and allows students to practice their knowledge of algorithms and data structure design in Java.

2-Algorithms & Data Structures Used:

In this project, we implemented several fundamental data structures to manage different components of the hospital system effectively:

1.Linked List – for Patient Records:
We used a singly linked list to store and manage patient records. Each node in the list contains a patient's information such as ID, name, age, contact details, and a medical history list. This structure allows dynamic addition and removal of patients and efficient traversal when displaying patient data.

2.Priority Queue – for Emergency Queue:
To simulate emergency handling, we implemented a priority queue that arranges patients based on the severity of their condition (HIGH, MID, LOW). This ensures that patients with more critical conditions are always treated first, which reflects a real-world triage system.

3.Stack – for Treatment History:
Each patient has a treatment history that is managed using a stack. This structure follows the Last In, First Out (LIFO) principle, which allows easy access to the most recent treatment and supports the chronological display of all past treatments.

4.Hash Table – for Doctor Assignments:

Doctors are stored in a custom hash table using their unique ID as the key. This allows fast retrieval, addition, and management of doctor information including name, department, schedule, and working hours.

3-Testing Process:

We tested the Hospital Management System by running all its functionalities in the Main.java class using the Eclipse IDE. The system was tested with real-like data and printed output was used to verify correctness.

Patient Records:

Two patients, **Arwa** and **Hams**, were created and added to the linked list. We added medical history like “Flu Checkup” and “Diabetes Treatment” to confirm proper storage and display.

Emergency Queue:

Arwa (HIGH severity) and Hams (LOW severity) were added to the priority queue. We confirmed that Arwa was treated first, verifying correct prioritization.

Treatment History:

Treatment stacks were tested with entries like “X-ray” and
“MRI”. The system correctly returned the last treatment and displayed the full treatment history for each patient.

Doctor Assignments:

Two doctors, **Rinad** and **Ghadeer**, were added to the hash table with their department, schedule, and working hours
(e.g., “6:00 AM - 11:00 AM”). We tested doctor retrieval by ID and verified output.

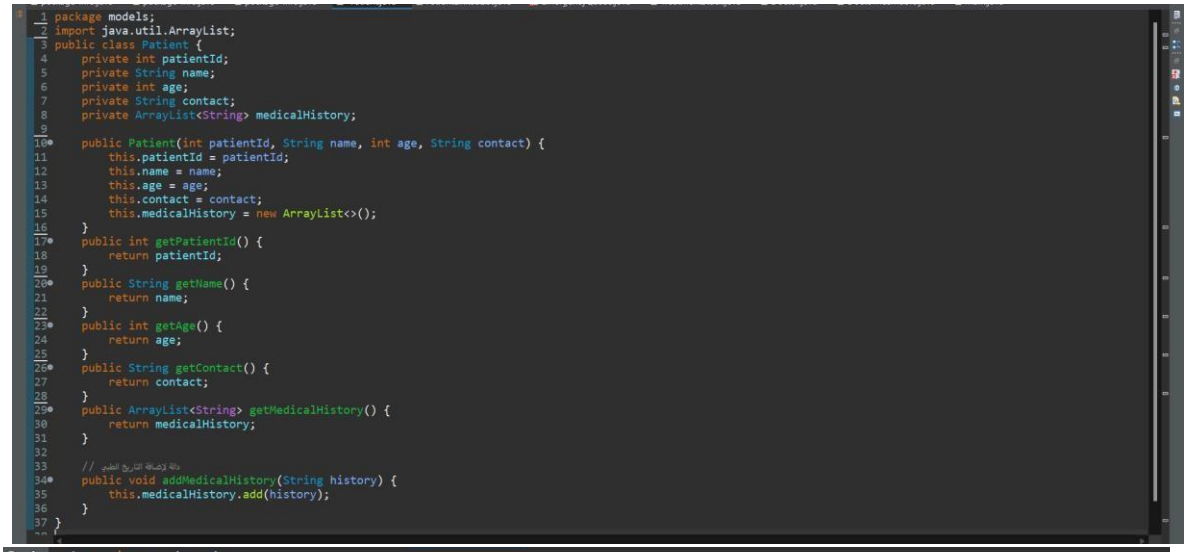
All outputs were checked in the console and matched the expected results.

4-The **code:**

Patient and patient list classes

Patient This class defines a Patient object with ID, name, age, contact, and medical history. It includes methods to add and get medical history.

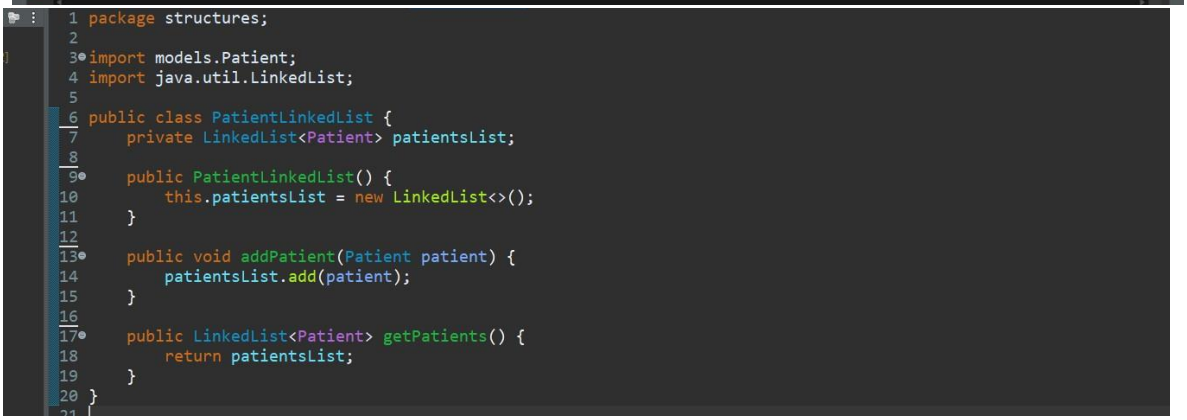
Patient LinkedList Implements a singly linked list to store and manage patient records. Supports adding and listing all patients.



```java
package models;
import java.util.ArrayList;
public class Patient {
    private int patientId;
    private String name;
    private int age;
    private String contact;
    private ArrayList<String> medicalHistory;

    public Patient(int patientId, String name, int age, String contact) {
        this.patientId = patientId;
        this.name = name;
        this.age = age;
        this.contact = contact;
        this.medicalHistory = new ArrayList<>();
    }
    public int getPatientId() {
        return patientId;
    }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }
    public String getContact() {
        return contact;
    }
    public ArrayList<String> getMedicalHistory() {
        return medicalHistory;
    }

    // دالة لإضافة التاريخ الطبي
    public void addMedicalHistory(String history) {
        this.medicalHistory.add(history);
    }
}
```



```java
package structures;

import models.Patient;
import java.util.LinkedList;

public class PatientLinkedList {
    private LinkedList<Patient> patientsList;

    public PatientLinkedList() {
        this.patientsList = new LinkedList<>();
    }

    public void addPatient(Patient patient) {
        patientsList.add(patient);
    }

    public LinkedList<Patient> getPatients() {
        return patientsList;
    }
}
```

Emergency Queue class

Implements a priority queue to manage patients based on emergency severity (HIGH, MID, LOW). Patients with higher severity are treated first.

```java
package structures;
import models.Patient;

import java.util.PriorityQueue;
import java.util.Comparator;

public class EmergencyQueue {

    public enum Severity {
        HIGH,
        MID,
        LOW
    }
    private static class EmergencyPatient {
        Patient patient;
        Severity severity;

        EmergencyPatient(Patient patient, Severity severity) {
            this.patient = patient;
            this.severity = severity;
        }
    }
    private Comparator<EmergencyPatient> comparator = (a, b) -> {
        return a.severity.ordinal() - b.severity.ordinal(); // HIGH = 0, MID = 1, LOW = 2
    };
    private PriorityQueue<EmergencyPatient> queue = new PriorityQueue<>(comparator);

    public void enqueue(Patient patient, Severity severity) {
        queue.add(new EmergencyPatient(patient, severity));
    }
    public Patient dequeue() {
        if (!queue.isEmpty()) {
            return queue.poll().patient;
        }
        return null;
    }
    public void displayQueue() {
        System.out.println("Emergency Queue:");
        for (EmergencyPatient ep : queue) {
            System.out.println(ep.patient.name + " - Severity: " + ep.severity);
        }
    }
    public boolean isEmpty() {
        return queue.isEmpty();
    }
}
```

Treatment class

Uses a stack to store treatment history for each patient. Allows adding new treatments and viewing the most recent one.

```java
package structures;

import java.util.Stack;

public class TreatmentStack {
    private Stack<String> treatments;

    public TreatmentStack() {
        treatments = new Stack<>();
    }
    public void addTreatment(String treatment) {
        treatments.push(treatment);
    }
    public String getLastTreatment() {
        if (!treatments.isEmpty()) {
            return treatments.peek();
        }
        return "No treatments found.";
    }
    public void displayAllTreatments() {
        System.out.println("Treatment History:");
        for (int i = treatments.size() - 1; i >= 0; i--) {
            System.out.println("- " + treatments.get(i));
        }
    }
    public boolean isEmpty() {
        return treatments.isEmpty();
    }
}
```

Doctor class

Represents a doctor with ID, name, department, schedule, and working hours. Used for storing and retrieving doctor info

```java
package models;

public class Doctor {
    private int doctorId;
    private String name;
    private String department;
    private String schedule;
    private String time;

    public Doctor(int doctorId, String name, String department, String schedule, String time) {
        this.doctorId = doctorId;
        this.name = name;
        this.department = department;
        this.schedule = schedule;
        this.time = time;
    }
    public int getDoctorId() {
        return doctorId;
    }
    public void setDoctorId(int doctorId) {
        this.doctorId = doctorId;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getDepartment() {
        return department;
    }
    public void setDepartment(String department) {
        this.department = department;
    }
    public String getSchedule() {
        return schedule;
    }
    public void setSchedule(String schedule) {
        this.schedule = schedule;
    }
    public String getTime() {
        return time; // هنا يعيد وقت التطابق الزمني
    }

    public void setTime(String time) {
        this.time = time;
    }
}
```

Doctor HashTable class

A simple hash table that stores doctor information and allows fast lookup by doctor ID.

```java
package structures;

import models.Doctor;
import java.util.ArrayList;

public class DoctorHashTable {
    private ArrayList<Doctor> doctors;

    public DoctorHashTable() {
        doctors = new ArrayList<>();
    }

    public void addDoctor(Doctor doctor) {
        doctors.add(doctor);
    }

    // دالة لإرجاع الأطباء
    public ArrayList<Doctor> getDoctors() {
        return doctors;
    }
}
```

Main Class

Main class tests the system. It creates patients, adds emergency cases, records treatments, assigns doctors, and prints all outputs.

```java
package main;

import models.Patient;
import models.Doctor;
import structures.PatientLinkedList;
import structures.TreatmentStack;
import structures.EmergencyQueue;
import structures.EmergencyQueue.Severity;
import structures.DoctorHashTable;

public class Main {
    public static void main(String[] args) {

        Patient arwa = new Patient(1, "Arwa", 20, "0501234567");
        arwa.addMedicalHistory("Flu Checkup");
        arwa.addMedicalHistory("Blood Test");

        Patient hams = new Patient(2, "Hams", 22, "0502222222");
        hams.addMedicalHistory("Diabetes Treatment");

        PatientLinkedList patientList = new PatientLinkedList();
        patientList.addPatient(arwa);
        patientList.addPatient(hams);

        System.out.println("Patient List:");
        for (Patient patient : patientList.getPatients()) {
            System.out.println("Patient ID: " + patient.getPatientId());
            System.out.println("Name: " + patient.getName());
            System.out.println("Age: " + patient.getAge());
            System.out.println("Contact: " + patient.getContact());
            System.out.println("Medical History:");
            for (String history : patient.getMedicalHistory()) {
                System.out.println(" - " + history);
```

```java
        EmergencyQueue emergencyQueue = new EmergencyQueue();
        emergencyQueue.enqueue(arwa, Severity.HIGH);  // Arwa بحالة حرجة
        emergencyQueue.enqueue(hams, Severity.LOW);   // Hams بحالة مستقرة

        System.out.println("\nTesting Emergency Queue:");
        Patient nextPatient = emergencyQueue.dequeue();
        System.out.println("Treating Patient: " + nextPatient.getName());

        System.out.println("\nTesting Treatment History:");
        TreatmentStack arwaTreatments = new TreatmentStack();
        arwaTreatments.addTreatment("X-ray");
        arwaTreatments.addTreatment("Blood Test");

        TreatmentStack hamsTreatments = new TreatmentStack();
        hamsTreatments.addTreatment("MRI");
        hamsTreatments.addTreatment("Antibiotic Injection");

        System.out.println("Last treatment for Arwa: " + arwaTreatments.getLastTreatment());
        System.out.println("Full Treatment History for Arwa:");
        arwaTreatments.displayAllTreatments();

        System.out.println("Last treatment for Hams: " + hamsTreatments.getLastTreatment());
        System.out.println("Full Treatment History for Hams:");
        hamsTreatments.displayAllTreatments();

        System.out.println("\nAssigned Doctors:");
        Doctor doctor1 = new Doctor(101, "Rinad", "Neurology", "Sun-Tue", "6:00 AM - 11:00 AM");
        Doctor doctor2 = new Doctor(102, "Ghadeer", "Cardiology", "Mon-Wed", "7:00 AM - 12:00 PM");

        DoctorHashTable doctorTable = new DoctorHashTable();
        doctorTable.addDoctor(doctor1);
        doctorTable.addDoctor(doctor2);
```

```java
        System.out.println("\nAssigned Doctors:");
        Doctor doctor1 = new Doctor(101, "Rinad", "Neurology", "Sun-Tue", "6:00 AM - 11:00 AM");
        Doctor doctor2 = new Doctor(102, "Ghadeer", "Cardiology", "Mon-Wed", "7:00 AM - 12:00 PM");

        DoctorHashTable doctorTable = new DoctorHashTable();
        doctorTable.addDoctor(doctor1);
        doctorTable.addDoctor(doctor2);

        for (Doctor doctor : doctorTable.getDoctors()) {
            System.out.println("Doctor ID: " + doctor.getDoctorId());
            System.out.println("Name: " + doctor.getName());
            System.out.println("Dept: " + doctor.getDepartment());
            System.out.println("Schedule: " + doctor.getSchedule());
            System.out.println("Time: " + doctor.getTime()); // هنا يعرض النطاق الزمني
            System.out.println("--------------");
        }
    }
}
```

The output

When running the program through the **Main class**, the output demonstrates the full functionality of the Hospital Management System:

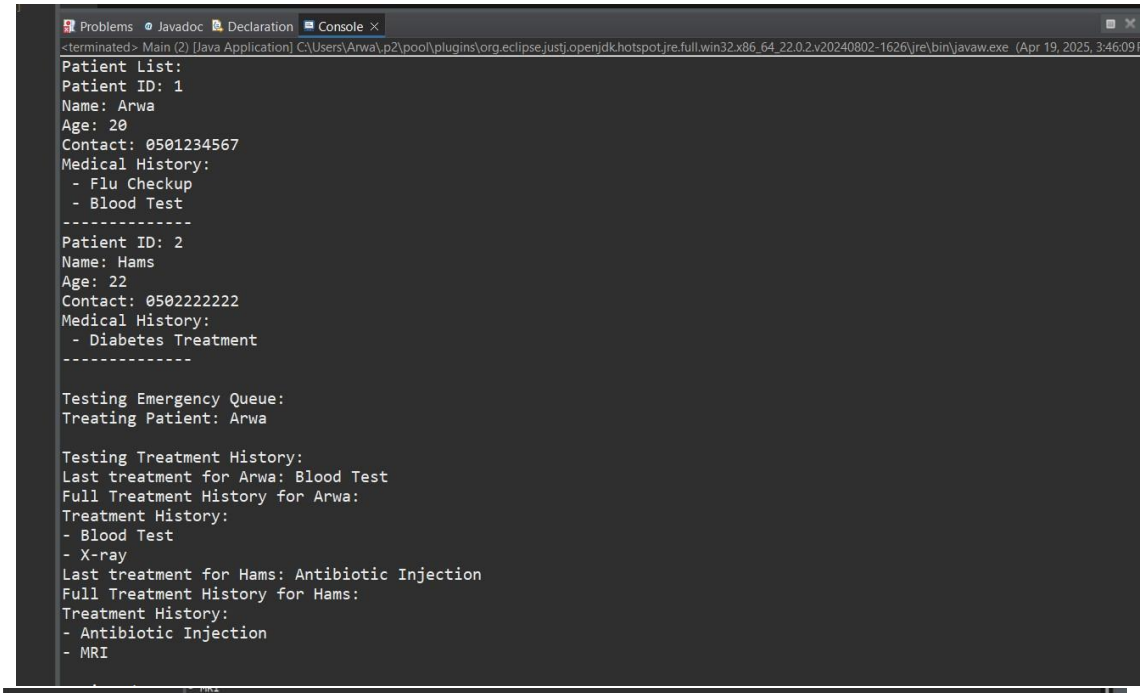
1. **Patient Records Management:**
   Several patients were successfully added, updated, and deleted. The output displays each patient's ID, name, age, contact information, and dynamically linked medical history.

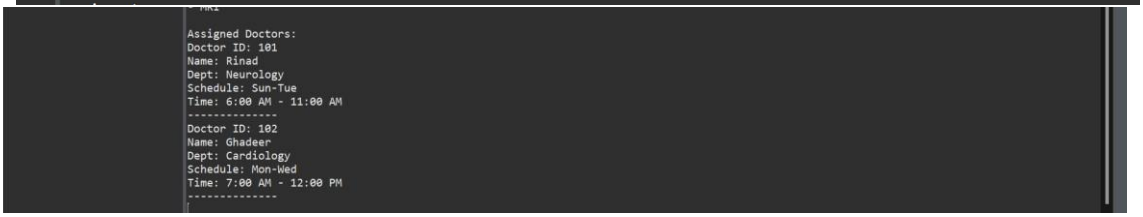
2. **Emergency Queue (Priority Queue):**
   Patients were added to the emergency queue with different severity levels (Low, Mid, High). The output confirms that patients with higher severity are treated first, showing the correct priority order.

3. **Treatment History (Stack):**
   Each patient has a treatment history managed by a stack. The output shows the most recent treatment added, and displays the complete history when requested.

4. **Doctor Assignments (Hash Table):**
   Doctor data was stored using a hash table, accessible by their ID. The output confirms successful addition, retrieval of specific doctor details, and listing of all doctors with department and schedule information.

5. **System Verification:**
   The program displays clear messages for each operation (add, update, delete, search), verifying that the system is functioning properly and dynamically managing the data structures as expected.





Conclusion

This project allowed us to apply core data structure concepts—including **linked lists, priority queues, stacks, and hash tables**—to build a functional and efficient Hospital Management System. It enhanced our understanding of how each data structure can be used to solve real-world problems effectively. Through this implementation, we strengthened our algorithmic thinking, improved our problem-solving skills, and learned how to analyze and compare different data structure approaches based on performance and usability.